



PROJECT REPORT

End-to-End Machine Learning Model Deployment using GitHub and Render

Heart Disease Risk Prediction — REST API & Cloud Deployment

Submitted by

Sushant Tiwari

VIT Bhopal University

Domain: Artificial Intelligence & Machine Learning
August 2026

1. Objective

The objective of this project is to build, evaluate, and deploy a machine learning model that predicts whether a patient is at risk of heart disease based on standard clinical parameters. The trained model is exposed through a REST API built with Flask, the complete project is version-controlled on GitHub, and the API is deployed as a live, publicly accessible web service using Render.

2. Dataset

Name: Heart Disease Prediction Dataset

Source (Kaggle): <https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset>

The dataset contains 14 columns describing a patient's clinical profile: age, sex, chest pain type (cp), resting blood pressure (trestbps), serum cholesterol (chol), fasting blood sugar (fbs), resting ECG results (restecg), maximum heart rate achieved (thalach), exercise-induced angina (exang), ST depression (oldpeak), slope of the peak exercise ST segment (slope), number of major vessels colored by fluoroscopy (ca), thalassemia (thal), and the binary target label indicating presence (1) or absence (0) of heart disease.

3. Task 1 — Data Understanding and Preprocessing

- Loaded the dataset using Pandas and inspected the first five records.
- Identified 13 numerical clinical features and one binary target variable.
- Verified there were no missing values in any column.
- Performed an 80% / 20% stratified train-test split to preserve class balance.
- Standardized all numerical features using scikit-learn's StandardScaler.

4. Task 2 — Model Development

A Random Forest Classifier was selected for this tabular classification problem, as it handles non-linear relationships between clinical features well and is robust to outliers without requiring extensive tuning.

Model Configuration

- Algorithm: Random Forest Classifier
- Estimators: 200 decision trees
- Max depth: 6 (to limit overfitting)
- Class weighting: balanced (to correct for the dataset's skew toward positive cases)
- Feature scaling: StandardScaler applied to all 13 input features

Evaluation

Metric	Value
Accuracy	75.6%
Precision (Heart Disease)	0.85
Recall (Heart Disease)	0.79
Precision (No Heart Disease)	0.59
Recall (No Heart Disease)	0.68

The trained model, fitted scaler, and the ordered feature list were serialized together into a single `model.pkl` file using `joblib`, ensuring the API always applies the exact same preprocessing at inference time as was used during training.

5. Task 3 — API Development

A REST API was developed using Flask with the following endpoints:

- GET / — a simple HTML form for manual browser testing
- GET /health — service and model health check
- POST /predict — accepts patient details as JSON and returns a JSON prediction

Example Request

```
POST /predict
Content-Type: application/json

{
  "age": 63, "sex": 1, "cp": 3, "trestbps": 145, "chol": 233,
  "fbs": 1, "restecg": 0, "thalach": 150, "exang": 0,
  "oldpeak": 2.3, "slope": 0, "ca": 0, "thal": 1
}
```

Example Response

```
{
  "prediction": "Heart Disease Detected",
  "prediction_label": 1,
  "probability": 0.6856
}
```

The API validates incoming JSON for missing fields, non-numeric values, and non-finite values (NaN / Infinity), and returns clear JSON error messages with appropriate HTTP status codes for invalid requests rather than crashing.

6. Task 4 — Cloud Deployment

Repository Structure

```
HeartDiseaseDeployment/
├── app.py                # Flask REST API
├── main.py               # direct launch file
├── train_model.py        # preprocessing + model training
├── model.pkl             # trained model + scaler (joblib)
├── heart.csv             # training dataset
├── requirements.txt
├── README.md
├── test_samples.json / .py
├── Procfile / render.yaml
├── .gitignore / .gitattributes
└── templates/index.html
```

Deployment Steps

- Pushed the complete project to a public GitHub repository.
- Created a new Web Service on Render and connected the GitHub repository.
- Configured the build command: `pip install -r requirements.txt`
- Configured the start command: `gunicorn app:app`
- Render built and deployed the service, exposing a public HTTPS URL.

Live Application URL

<https://<your-render-app-name>.onrender.com> (to be filled in after deployment — see README.md)

7. Results Summary

Metric	Value
Model Accuracy	75.6%
API Response Format	JSON
Deployment Platform	Render (Web Service)
Version Control	GitHub

8. Conclusion

The Random Forest model achieved 75.6% accuracy on the held-out test set, with balanced precision and recall across both classes after applying class-weighting to correct for the dataset's skew toward positive cases. Performance could likely be improved further with hyperparameter tuning or additional feature engineering, but the model is adequate for demonstrating a full deployment pipeline. The main challenges during deployment involved correctly packaging the scaler and feature order alongside the model so that raw JSON inputs are transformed identically to training data, and configuring Render's build and start commands (gunicorn instead of Flask's development server) for a stable, publicly reachable service. This project highlighted why MLOps practices matter: version-controlling code and model artifacts, automating preprocessing inside the API, and validating inputs all ensure that a model behaves consistently and reliably once it leaves a notebook and is exposed as a real, continuously running service.

9. Learning Outcomes

- Built and evaluated a machine learning classification model on real-world-style clinical data.
- Saved and loaded trained models using joblib.
- Developed a REST API using Flask with proper input validation and error handling.
- Managed project code using GitHub with a clean, deployment-ready repository structure.
- Deployed a machine learning application to the cloud using Render.
- Understood the fundamentals of MLOps — model packaging, version control, deployment, and serving predictions through an API.

10. References

1. Dataset — <https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset>
2. Flask Documentation — <https://flask.palletsprojects.com>
3. scikit-learn Documentation — <https://scikit-learn.org>
4. Render Documentation — <https://render.com/docs>